CRYSTALS-Dilithium (Dilithium-2 / ML-DSA-44) Implementation in C Using NTT

Technical Report, Mathematical Derivations, Interactive Walkthroughs, and Live Demonstration Guide

Companion Engineering Documentation for `Dilithium_c.c`

Abstract

This report provides a comprehensive mathematical and systems-level analysis of the standalone C implementation of **CRYSTALS-Dilithium (Dilithium-2 / NIST ML-DSA-44)** in `Dilithium_c.c`. Designed as a companion to the Kyber-512 PKE report for technical presentations, it explains the transition from encryption to lattice-based digital signatures via **Fiat-Shamir with Aborts** over $R_q = \mathbb{Z}_{8380417}[X]/(X^{256}+1)$. We derive the 32-bit Montgomery reduction (58728449), the complete 8-layer NTT ($\zeta = 1753$), division-free constant-time `HighBits`/`LowBits` decomposition (11275), rejection sampling security bounds, a concrete numerical walkthrough of signing and verification at coefficient index 0, and four live code experiments.

1 Executive Overview and Codebase Architecture

While CRYSTALS-Kyber (ML-KEM) establishes shared secrets, **CRYSTALS-Dilithium** (standardized by NIST as **ML-DSA** in FIPS 204) provides post-quantum digital signatures based on the simultaneous hardness of the **Module Learning With Errors (M-LWE)** and **Module Short Integer Solution (M-SIS)** problems. Unlike encryption, a naive lattice signature $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$ would leak the secret key $\mathbf{s}_1$ through statistical correlation unless $\mathbf{z}$ is made independent of $\mathbf{s}_1$ via **Rejection Sampling (Fiat-Shamir with Aborts)**. Table 1 maps every C function in `Dilithium_c.c` to its mathematical role.

Table 1: Structural Map of `Dilithium_c.c` Functions and Primitives

C Function / Symbol	Lines	Mathematical & Systems Role
`DILITHIUM_Q`, `K`, `L`, `GAMMA1/2`	13–21	Prime $q = 8,380,417$, dimensions $(k,\ell) = (4,4)$, $\eta = 2$, $\tau = 39$, $\beta = 78$, $\gamma_1 = 2^{17}$, $\gamma_2 = 95,232$.
`bitrev8`, `prng32`, `challenge_prng`	27–61	8-bit reversal for 256 twiddle factors and Xoshiro128** PRNGs for randomness and hashing.
`init_zetas`	64–79	Precomputes 256 bit-reversed powers of $\zeta = 1753$ scaled by $R = 2^{32} \equiv 4,193,792 \pmod q$.
`montgomery_reduce`, `fqmul`	82–101	64-to-32-bit reduction computing $a \cdot 2^{-32} \bmod q$ using $q^{-1} \bmod 2^{32} = 58,728,449$.
`reduce32`, `caddq`	88–97	Centered reduction $a - \lfloor (a + 2^{22})/2^{23} \rfloor q$ and branchless addition of q if negative.
`ntt`, `invntt`	104–138	Complete 8-layer Cooley-Tukey forward NTT and Gentleman-Sande inverse NTT ($f = 41,978$).
`poly_pointwise_montgomery`	141–145	Pure 256-element scalar Montgomery multiplication $\hat{a}_i \cdot \hat{b}_i \cdot R^{-1} \bmod q$ (no `basemul` pairs).
`decompose`, `poly_decompose`	168–186	Constant-time branchless rounding $a = r_1(2\gamma_2) + r_0$ into HighBits (r_1) and LowBits (r_0).
`poly_uniform_eta`, `gamma1`	189–205	Samples secret keys in $[-\eta, \eta] = [-2, 2]$ and masking vectors $\mathbf{y}$ in $[-\gamma_1 + 1, \gamma_1]$.
`poly_challenge`, `poly_chknorm`	207–239	Inside-out Fisher-Yates `SampleInBall` ($\tau = 39$ signs) and branchless $\|\cdot\|_\infty$ norm checker.
`Dilithium_KeyGen`, `Sign`, `Verify`	250–380	Core ML-DSA key generation, Fiat-Shamir with Aborts signing loop, and verification.

2 Ring Parameters, Complete Splitting, and Comparison with Kyber

Both Kyber and Dilithium operate over $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$, but Dilithium requires a much larger modulus $q = 8,380,417 = 2^{23} - 2^{13} + 1$ to accommodate the masking polynomial $\mathbf{y} \in [-2^{17}+1, 2^{17}]$ needed to hide secret keys during signing. Table 2 highlights the architectural differences between `Kyber_c.c` and `Dilithium_c.c`.

3 32-Bit Modular Arithmetic and Complete 8-Layer NTT

3.1 32-Bit Montgomery Reduction (58728449) and Centered Reduction (`reduce32`)

Because $q = 2^{23} - 2^{13} + 1$, its modular inverse modulo $R = 2^{32}$ satisfies a remarkable algebraic identity:

$$\left(2^{23} - 2^{13} + 1\right)\left(7 \cdot 2^{23} + 2^{13} + 1\right) = 8,380,417 \times 58,728,449 = 7 \cdot 2^{46} - 6 \cdot 2^{36} + 1 \equiv 1 \pmod{2^{32}} \tag{1}$$

Table 2: Architectural Comparison: `Kyber_c.c` (ML-KEM) vs. `Dilithium_c.c` (ML-DSA)

Feature	Kyber-512 (`Kyber_c.c`)	Dilithium-2 (`Dilithium_c.c`)	Engineering Reason for Difference
Modulus q	$3,329 = 13 \cdot 2^8 + 1$	$8,380,417 = 2^{23} - 2^{13} + 1$	Dilithium needs room for $\gamma_1 = 2^{17}$ masking vectors.
Data Width	`int16_t` / `int32_t`	`int32_t` / `int64_t`	$q \approx 2^{23}$ fits in 32-bit registers with 8 bits of headroom.
NTT Depth	7 layers (len : $128 \to 2$)	8 layers (len : $128 \to 1$)	$512 \mid (8380417 - 1)$, so $X^{256} + 1$ splits into 256 linear factors.
Root of Unity	$\zeta = 17$ (256-th root)	$\zeta = 1753$ (512-th root)	$1753^{256} \equiv -1 \pmod{8380417}$.
NTT Multiply	`basemul` (128 pairs)	`fqmul` (256 scalars)	Complete splitting eliminates degree-1 cross-multiplication!
Montgomery R	$R = 2^{16}$, $q^{-1} \equiv 62209$	$R = 2^{32}$, $q^{-1} \equiv 58728449$	Shifts right by 32 bits (`>> 32`) on 64-bit products.
InvNTT Factor	$f = 1441 \equiv R^2/128$	$f = 41978 \equiv R^2/256$	Divides by $2^8 = 256$ layers and cancels R^{-1} from `fqmul`.

In `montgomery_reduce(int64_t a)` (lines 82–86), multiplying a 64-bit product a by $58,728,449$ mod 2^{32} yields $t = (a \cdot q^{-1}) \bmod 2^{32}$. Subtracting $t \cdot q$ zeroes the lower 32 bits of a, so `>> 32` computes $a \cdot 2^{-32} \bmod q$ without division.

Similarly, because $q = 2^{23} - 8191$ is within 0.1% of 2^{23}, `reduce32(int32_t a)` (lines 88–92) estimates the quotient a/q simply by shifting right by 23 bits with rounding bias 2^{22}:

$$t = (a + 2^{22}) \gg 23, \qquad \texttt{reduce32}(a) = a - 8380417 \cdot t \in [-4190208, 4190208] \tag{2}$$

3.2 Complete 8-Layer NTT, Pointwise Multiplication, and `invntt` Scaling

Since $q - 1 = 8,380,416 = 512 \times 16,368$, $\zeta = 1753$ is a primitive 512-th root of unity ($1753^{256} \equiv -1 \pmod q$). Thus, $X^{256} + 1$ factors completely across **all 8 binary tree layers** (`len` $= 128, 64, 32, 16, 8, 4, 2, 1$) into 256 linear terms:

$$R_q \cong \prod_{i=0}^{255} \mathbb{Z}_{8380417}[X] / \left(X - \zeta^{2\text{br}_8(i)+1}\right) \tag{3}$$

Consequently, `poly_pointwise_montgomery` (lines 141–145) requires **no degree-1 `basemul` cross-products**: multiplying two NTT-domain polynomials is just 256 scalar calls `r[i] = fqmul(a[i], b[i])`.

In `invntt()` (lines 119–138), traversing `zetas[k--]` from $k = 255$ down to 1 fetches $\zeta^{256-m} \equiv -\zeta^{-m} \pmod q$. Just as in Kyber, line 129 reverses the butterfly subtraction (`r[j + len] = r[j + len] - t`) to cancel the minus sign, and line 136 multiplies by $f = 41,978$. Since $256 \times 41,978 = 10,746,368 = q + 2,365,951 \equiv R^2 \pmod q$, `fqmul(r[j], 41978)` multiplies by $\frac{R^2}{256} \cdot R^{-1} = \frac{R}{256} \bmod q$, simultaneously dividing by 256 and canceling the R^{-1} factor introduced by `poly_pointwise_montgomery`.

4 Fiat-Shamir with Aborts and Constant-Time Decomposition

4.1 Division-Free `HighBits` and `LowBits` Decomposition (`decompose`)

In Dilithium, the verifier must reconstruct the high-order commitment $\mathbf{w}_1 = \text{HighBits}(\mathbf{w})$ from the noisy relation $\mathbf{Az} - c\mathbf{t} = \mathbf{w} - c\mathbf{s}_2$. Every coefficient $a \in [0, q-1]$ is decomposed modulo $2\gamma_2 = 190,464 = 128 \times 1488$ as:

$$a \equiv r_1 \cdot (2\gamma_2) + r_0 \pmod q, \qquad r_0 \in [-\gamma_2, \gamma_2], \quad r_1 \in [0, 43] \tag{4}$$

where $q - 1 = 44 \times (2\gamma_2)$. Lines 168–179 implement this in constant time without division:

1. **Fixed-Point Quotient (`11275`):** First divide by 128 via `a1 = (a + 127) >> 7`. Then divide by 1488 with nearest-integer rounding using $\lceil 2^{24}/1488 \rceil = 11,275$: `a1 = (a1 * 11275 + (1 << 23)) >> 24`.
2. **Branchless Wrap-Around at $q - 1$:** When $a \in [q - \gamma_2, q - 1]$, a_1 reaches 44. Since $44 \cdot (2\gamma_2) = q - 1 \equiv -1 \pmod q$, this top bin belongs to the $r_1 = 0$ residue class around zero! Line 174 executes `a1 ^= ((43 - a1) >> 31) & a1`: if $a_1 = 44$, then $43 - 44 = -1$, whose arithmetic shift `>> 31` is `0xFFFFFFFF`, XORing a_1 with itself to zero it out without a branch.
3. **Remainder Centering:** Line 176 subtracts q whenever $a_0 > (q-1)/2$ using the mask `(((Q - 1)/2 - a0) >> 31) & Q`.

4.2 Why the Two Rejection Checks Guarantee Security and Zero-Hint Correctness

During `Dilithium_Sign` (lines 285–347), the signer samples $\mathbf{y} \in [-\gamma_1 + 1, \gamma_1]^{4 \times 256}$, computes $\mathbf{w} = \mathbf{Ay}$, hashes $(\text{msg}, \mathbf{w}_1)$ into a sparse challenge c having $\tau = 39$ coefficients in $\{-1, +1\}$ (`poly_challenge`), and computes

$\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$. Since $\|s_1\|_\infty, \|s_2\|_\infty \leq \eta = 2$ and c has 39 non-zero entries of ± 1, the products satisfy $\|c\mathbf{s}_1\|_\infty \leq 39 \times 2 = 78 = \beta$ and $\|c\mathbf{s}_2\|_\infty \leq 78 = \beta$. The loop aborts and restarts unless two conditions hold:

Check 1: Zero-Knowledge Security ($\|\mathbf{z}\|_\infty < \gamma_1 - \beta = 130,994$, line 326): Regardless of the specific value of $c\mathbf{s}_1 \in [-\beta, \beta]$, every value inside $[-\gamma_1 + 1 + \beta, \gamma_1 - \beta]$ is reached by exactly one $\mathbf{y} = \mathbf{z} - c\mathbf{s}_1 \in [-\gamma_1 + 1, \gamma_1]$. Rejecting outside this box makes the distribution of **z uniform and completely independent of** $\mathbf{s}_1$.

Check 2: Carry-Free Correctness ($\|\mathbf{LowBits}(\mathbf{w} - c\mathbf{s}_2)\|_\infty < \gamma_2 - \beta = 95,154$, line 339): In `Dilithium_Verify`, the verifier computes $\mathbf{Az} - c\mathbf{t} = \mathbf{A}(\mathbf{y} + c\mathbf{s}_1) - c(\mathbf{As}_1 + \mathbf{s}_2) = \mathbf{w} - c\mathbf{s}_2$. Since $\|c\mathbf{s}_2\|_\infty \leq \beta$, ensuring that the low bits of $\mathbf{w} - c\mathbf{s}_2$ lie strictly within $(-\gamma_2 + \beta, \gamma_2 - \beta)$ guarantees that subtracting $c\mathbf{s}_2$ never crosses a $\pm\gamma_2$ rounding boundary. Therefore, HighBits($\mathbf{Az} - c\mathbf{t}$) = HighBits($\mathbf{w} - c\mathbf{s}_2$) = $\mathbf{w}_1$ holds identically!

5 Interactive Numerical Walkthrough: Tracing Coefficient Index 0

Let us trace concrete integer values through the exact C expressions in `Dilithium_c.c`:

Part A: Tracing 32-Bit Montgomery Multiplication `fqmul(-3188622, 12000)`

1. For $i = 1$, $\text{br}_8(1) = 128$, and `zetas[1]` $= 1753^{128} \cdot 2^{32} \bmod 8380417 = -3,188,622$. Multiplying by $b = 12,000$ gives 64-bit product $a = -38,263,464,000$.
2. Inside `montgomery_reduce`: `t = (int32_t)((uint64_t)a * 58728449ULL)` $= -514,496,096$.
3. Subtracting $t \cdot q$: $a - t \cdot 8,380,417 = -38,263,464,000 - (-4,311,691,829,394,432) = 4,311,653,565,930,432$.
4. Shifting right by 32 bits (`>> 32`) divides by 2^{32} with zero remainder, returning $1,003,885$ (matching $1753^{128} \times 12000 \equiv 1,003,885 \pmod{8380417}$).

Part B: Tracing Signing, Decomposition, and Verification at Index 0

Stage 1: Commitment (`Dilithium_Sign`): Suppose at row 0, index 0, masking $\mathbf{y}$ produces $w[0][0] = 3,500,120$.

Stage 2: Decompose (`decompose`): `a1` $= (3500120 + 127) \gg 7 = 27,345$; `a1` $= (27345 \times 11275 + 2^{23}) \gg 24 = 316,703,483 \gg 24 = 18$. Then `a0` $= 3,500,120 - 18 \times 190,464 = 3,500,120 - 3,428,352 = +71,768$. Thus $w_1[0][0] = 18$ and $w_0[0][0] = +71,768 \in [-95232, 95232]$.

Stage 3: Response & Rejection Checks: Suppose $y[0][0] = -42,500$, $(c\mathbf{s}_1)[0][0] = +34$, and $(c\mathbf{s}_2)[0][0] = -29$ (both within $[-\beta, \beta] = [-78, 78]$).

- Response: $z[0][0] = -42,500 + 34 = -42,466$. Check 1: $|-42,466| < 130,994$ (**PASS**).
- LowBits check: $w[0][0] - (c\mathbf{s}_2)[0][0] = 3,500,120 - (-29) = 3,500,149 = 18 \times 190,464 + 71,797$. Thus $r_0 = +71,797$. Check 2: $|71,797| < 95,154$ (**PASS**).

Stage 4: Verification (`Dilithium_Verify`): Verifier computes $(\mathbf{Az} - c\mathbf{t})[0][0] = w[0][0] - (c\mathbf{s}_2)[0][0] = 3,500,149$. Calling `decompose` yields `a1` $= (3500149 + 127) \gg 7 = 27,345 \implies w_1'[0][0] = 18 = w_1[0][0]$. Re-hashing $(\text{msg}, \mathbf{w}_1')$ reproduces the exact challenge $c' == c$!

6 Live Code Experiments, Benchmark Analysis, and Presentation Q&A

6.1 Four Interactive Code Experiments for Live Demonstrations

Exp 1: Disable the LowBits Rejection Check (Line 339): Comment out `if (poly_chknorm(r0_tmp, ...)) reject = 1;` in `Dilithium_Sign`. Over 500 benchmark runs, verification intermittently **FAILS** whenever $w_0 - c\mathbf{s}_2$ crosses $\pm\gamma_2 = \pm 95,232$, shifting w_1' by ± 1 and causing $c' \neq c$.

Exp 2: Prove the Center-Reduction Requirement on $c\mathbf{s}_1$ (Line 324): Comment out `poly_reduce(sig_z[j]);` before `poly_add(sig_z[j], sig_z[j], y[j])`. Negative coefficients of $c\mathbf{s}_1$ remain near $q \approx 8.38 \times 10^6$, causing `poly_chknorm` to reject every attempt and hang in an infinite abort loop!

Exp 3: Test the $q - 1$ Boundary Wrap in `decompose` (Line 174): Comment out `a1 ^= ((43 - a1) >> 31) & a1;`. Whenever a coefficient lands in $[q - \gamma_2, q - 1]$, a_1 becomes 44 instead of wrapping to 0, breaking verification.

Exp 4: Scale to Dilithium-3 / ML-DSA-65 (Lines 15–16): Change `DILITHIUM_K` to `6` and `DILITHIUM_L` to `5` to benchmark NIST Security Level 3 in real time.

6.2 Hardware Cycle Benchmark Breakdown (`__rdtsc`)

6.3 Anticipated Presentation Q&A Reference

- **Q: Why is `Dilithium_Verify` so much faster than `Dilithium_Sign`?** Signing is probabilistic and runs the Fiat-Shamir rejection loop $M \approx 4.25$ times on average until both $\|\mathbf{z}\|_\infty$ and $\|\mathbf{r}_0\|_\infty$ bounds pass, whereas verification is a single deterministic pass with zero PRNG sampling.

Table 3: Algorithmic Complexity Per Execution in `Dilithium_c.c` ($k = 4, \ell = 4$, Avg Attempts ≈ 4.25)

Operation	NTTs	InvNTTs	Pointwise	Performance Profile & Dominant Cost
`ntt(r)` / `invntt(r)`	1 / 0	0 / 1	0	$256 \times 8 = 2,048$ butterfly steps; 32-bit registers avoid overflow reductions inside `invntt`.
`Dilithium_KeyGen`	8	4	16	Reject-samples $4 \times 4 \times 256 = 4,096$ uniform entries for $\hat{\mathbf{A}}$ and computes $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$.
`Dilithium_Sign`	$5M$	$12M$	$24M$	Runs $M \approx 4.25$ iterations on average; each attempt executes 4 **y**-NTTs, 4 **w**-InvNTTs, 4 **z**-InvNTTs, and 4 $c\mathbf{s}_2$-InvNTTs.
`Dilithium_Verify`	9	4	20	**Single-pass deterministic:** 4 **z**-NTTs, 1 c-NTT, 4 **t**-NTTs, and 4 ($\mathbf{Az} - c\mathbf{t}$)-InvNTTs ($\sim 4\times$ faster than Sign).

- **Q: Why does full FIPS 204 include a 1-bit `MakeHint`/`UseHint` vector h?** To shrink the public key **t** by dropping its low 13 bits ($\mathbf{t}_0$), which adds a small term $c\mathbf{t}_0$ during verification; our implementation stores full **t**, making HighBits($\mathbf{Az} - c\mathbf{t}$) $= \mathbf{w}_1$ exact without hints.
- **Q: Compilation Command:** `gcc -O3 -march=native -Wall -Wextra Dilithium_c.c -o dilithium_bench && ./dilithium_bench`